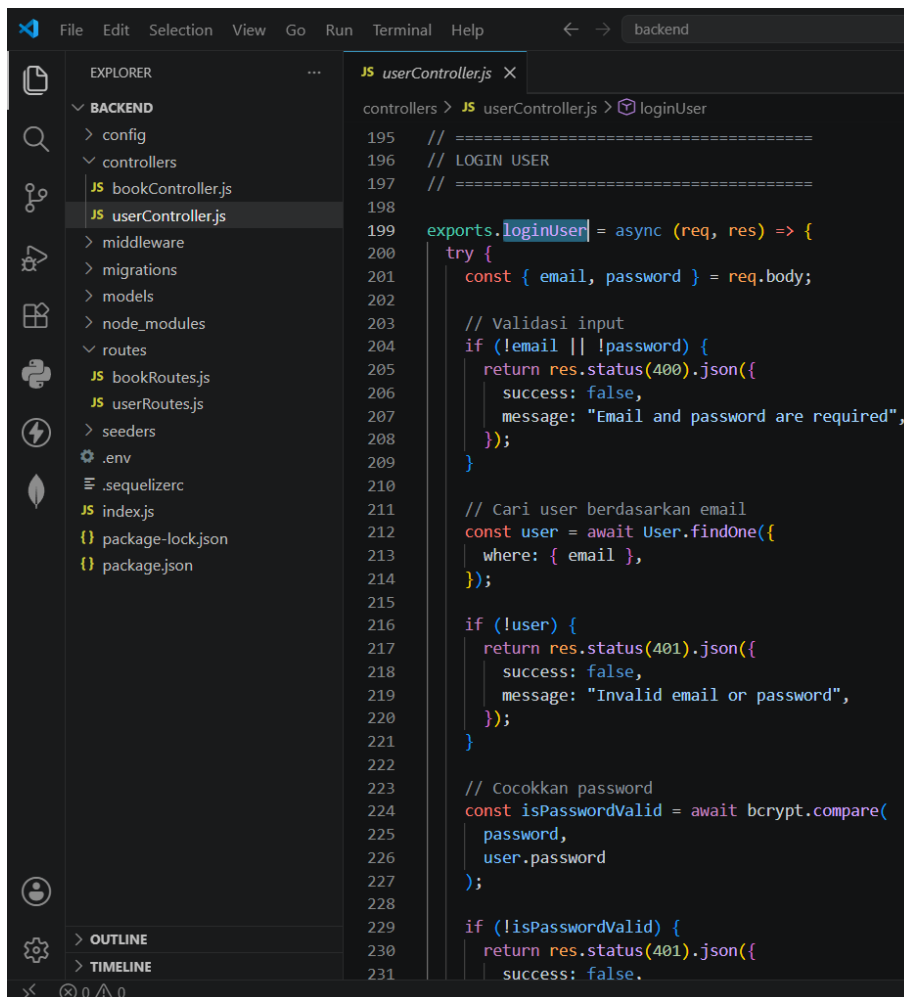


Nama : Rafly Maulana Yusuf
NPM : 242310014
Kelas : TI-24-KA
Mata Kuliah : Lab. Pemrograman Web

BAB VIII AUTHENTICATION AND INTEGRATING

1. Backend Authentication

SS 1 backend/controllers/userController.js

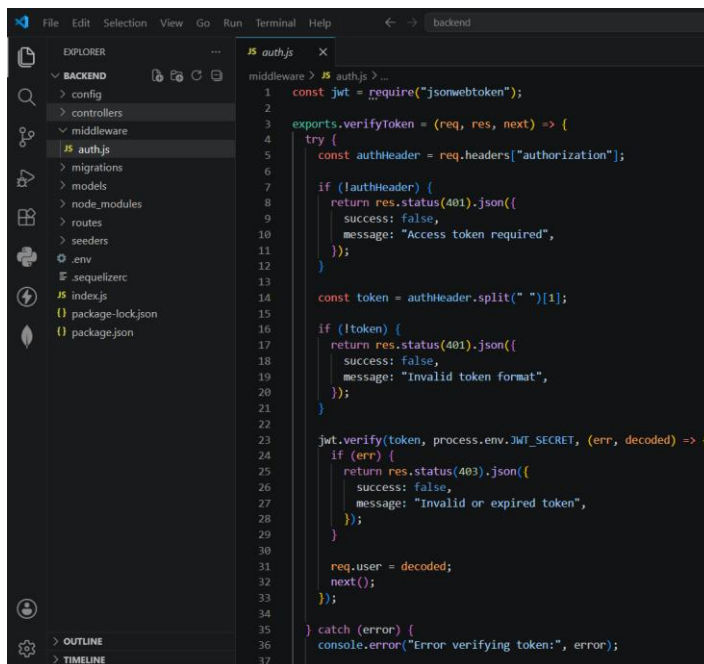


```
195 // =====
196 // LOGIN USER
197 // =====
198
199 exports.loginUser = async (req, res) => {
200   try {
201     const { email, password } = req.body;
202
203     // Validasi input
204     if (!email || !password) {
205       return res.status(400).json({
206         success: false,
207         message: "Email and password are required",
208       });
209     }
210
211     // Cari user berdasarkan email
212     const user = await User.findOne({
213       where: { email },
214     });
215
216     if (!user) {
217       return res.status(401).json({
218         success: false,
219         message: "Invalid email or password",
220       });
221     }
222
223     // Cocokkan password
224     const isPasswordValid = await bcrypt.compare(
225       password,
226       user.password
227     );
228
229     if (!isPasswordValid) {
230       return res.status(401).json({
231         success: false,
```

Penjelasan:

Menambahkan fungsi login menggunakan bcrypt untuk verifikasi password dan JWT untuk menghasilkan access token yang digunakan pada proses autentikasi.

SS 2 backend/middleware/auth.js

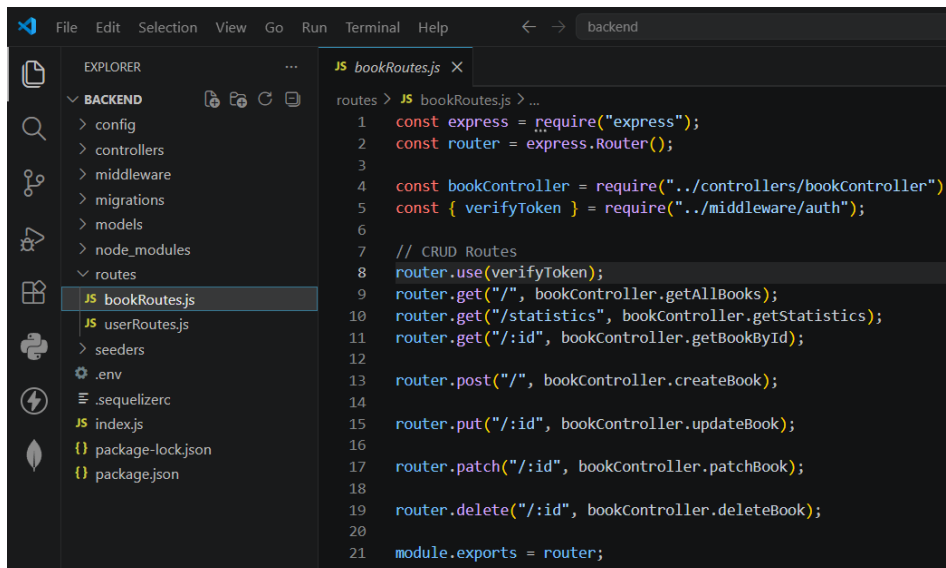


```
1 const jwt = require("jsonwebtoken");
2
3 exports.verifyToken = (req, res, next) => {
4   try {
5     const authHeader = req.headers["authorization"];
6
7     if (!authHeader) {
8       return res.status(401).json({
9         success: false,
10        message: "Access token required",
11      });
12    }
13
14    const token = authHeader.split(" ")[1];
15
16    if (!token) {
17      return res.status(401).json({
18        success: false,
19        message: "Invalid token format",
20      });
21    }
22
23    jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
24      if (err) {
25        return res.status(403).json({
26          success: false,
27          message: "Invalid or expired token",
28        });
29      }
30
31      req.user = decoded;
32      next();
33    });
34  } catch (error) {
35    console.error("Error verifying token:", error);
36  }
37 }
```

Penjelasan:

Middleware verifyToken digunakan untuk memverifikasi JWT sebelum pengguna dapat mengakses endpoint yang bersifat protected.

SS 3 backend/routes/bookRoutes.js



```
1 const express = require("express");
2 const router = express.Router();
3
4 const bookController = require("../controllers/bookController");
5 const { verifyToken } = require("../middleware/auth");
6
7 // CRUD Routes
8 router.use(verifyToken);
9 router.get("/", bookController.getAllBooks);
10 router.get("/statistics", bookController.getStatistics);
11 router.get("/:id", bookController.getBookById);
12
13 router.post("/", bookController.createBook);
14
15 router.put("/:id", bookController.updateBook);
16
17 router.patch("/:id", bookController.patchBook);
18
19 router.delete("/:id", bookController.deleteBook);
20
21 module.exports = router;
```

Penjelasan:

Seluruh endpoint books diamankan menggunakan middleware verifyToken sehingga hanya pengguna yang telah login yang dapat mengaksesnya.

SS 4 backend/routes/userRoutes.js

The screenshot shows the VS Code interface with the Explorer view on the left and the Editor view on the right. The Explorer view displays the project structure, with the following files and folders listed:

- BACKEND
 - config
 - controllers
 - middleware
 - migrations
 - models
 - node_modules
 - routes
 - bookRoutes.js
 - userRoutes.js**
 - seeders
 - .env
 - .sequelizerc
 - index.js
 - package-lock.json
 - package.json

The Editor view shows the code for `userRoutes.js`, which defines routes for a user authentication system using Express.js. The code is as follows:

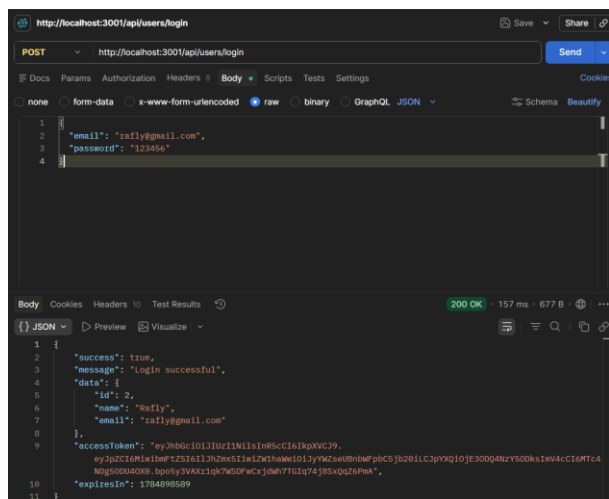
```
routes > JS userRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3
4  const userController = require("../controllers/userController");
5  const { verifyToken } = require("../middleware/auth");
6
7  // =====
8  // PUBLIC ROUTES
9  // =====
10
11 router.post("/login", userController.loginUser);
12 router.post("/", userController.createUser);
13
14 // =====
15 // PROTECTED ROUTES
16 // =====
17
18 router.use(verifyToken);
19
20 router.get("/", userController.getAllUsers);
21 router.get("/:id", userController.getUserById);
22 router.put("/:id", userController.updateUser);
23 router.delete("/:id", userController.deleteUser);
24
25 module.exports = router;
```

Penjelasan:

Endpoint login dan register bersifat public, sedangkan endpoint CRUD user memerlukan autentikasi JWT.

2. Testing Postman

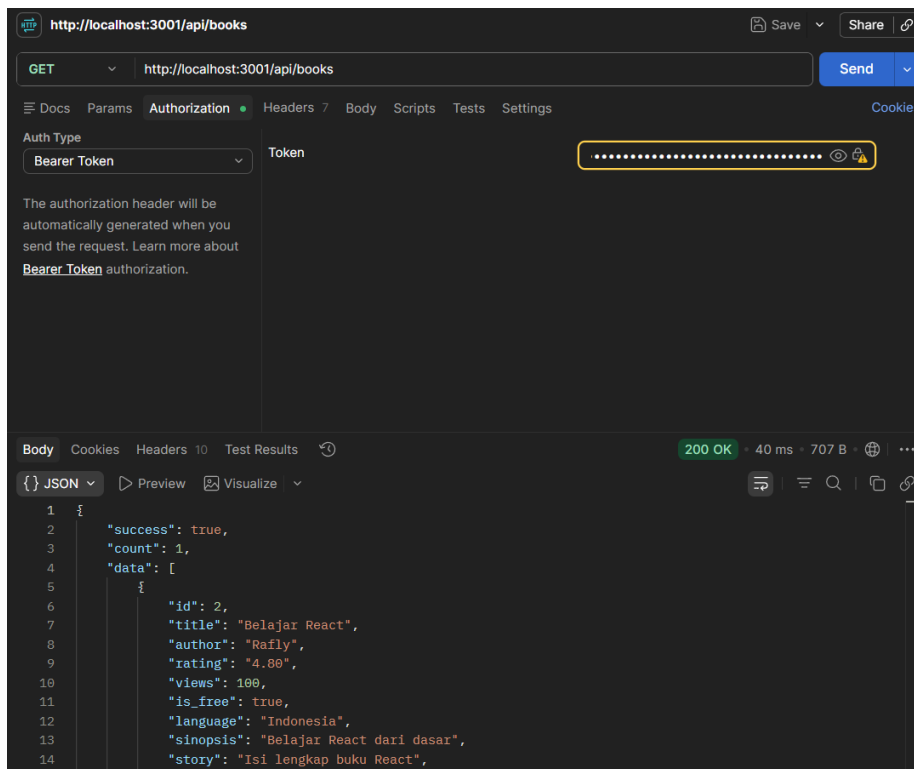
SS 5 POST /api/users/login



Penjelasan:

Pengguna berhasil login dan memperoleh accessToken yang akan digunakan sebagai Bearer Token pada request selanjutnya.

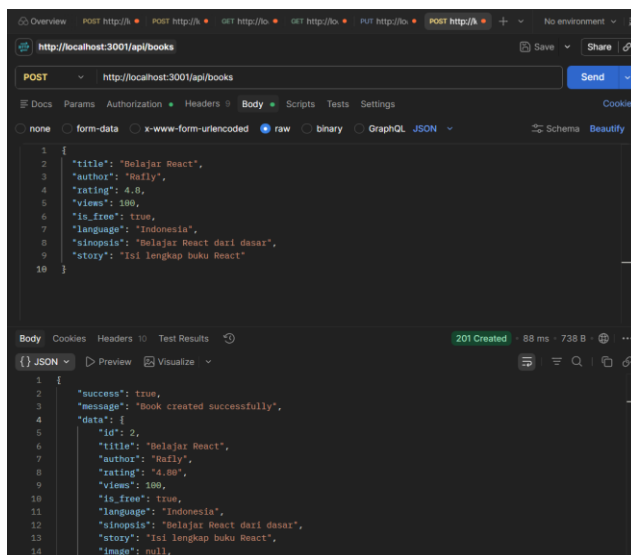
SS 6 GET Books



Penjelasan:

Endpoint books berhasil diakses menggunakan Bearer Token.

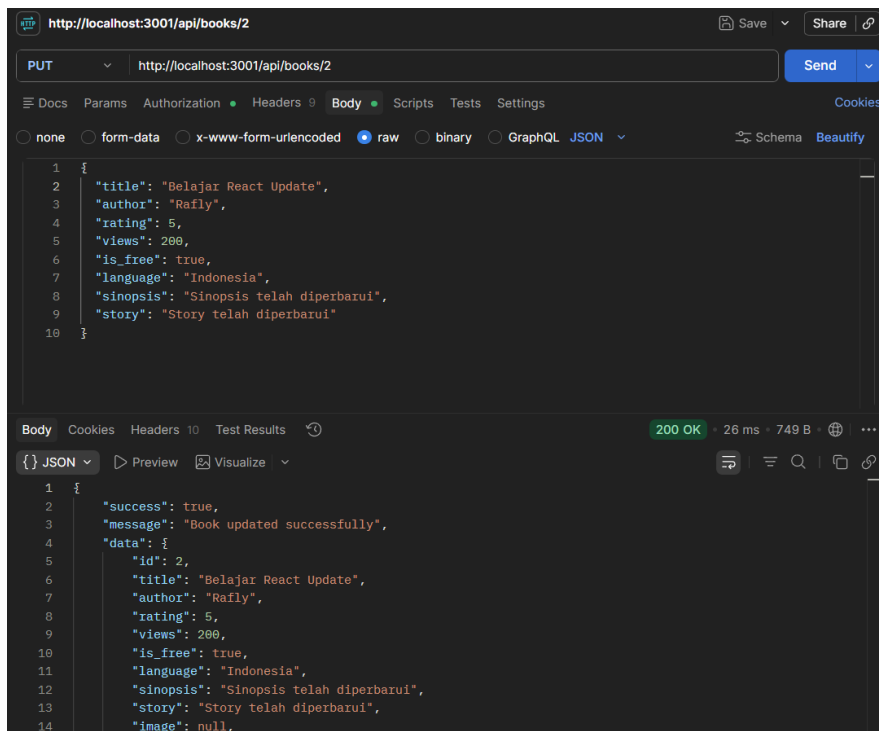
SS 7 POST Book



Penjelasan:

Data buku berhasil ditambahkan ke database melalui REST API.

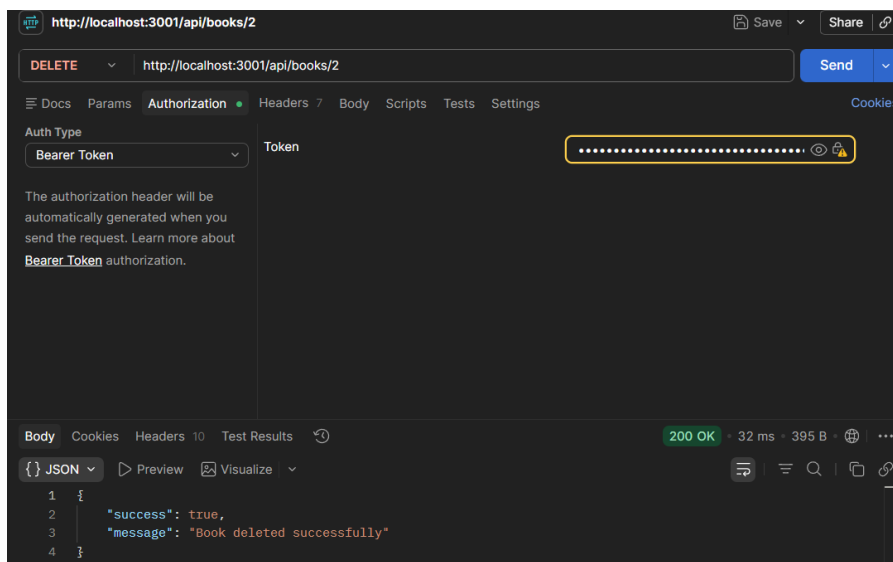
SS 8 PUT Book



Penjelasan:

Data buku berhasil diperbarui menggunakan endpoint update.

SS 9 DELETE Book

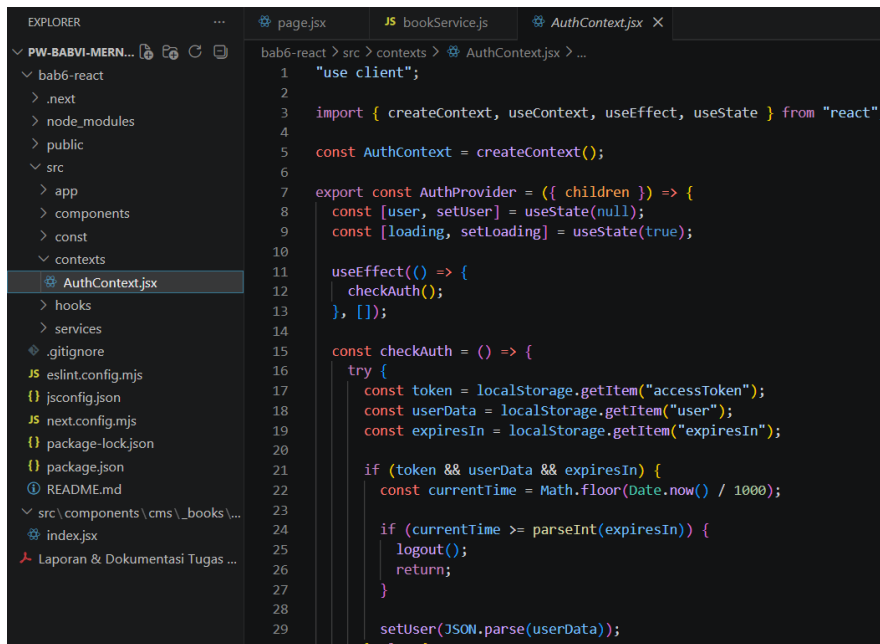


Penjelasan:

Data buku berhasil dihapus melalui endpoint delete.

3. Frontend

SS 10 src/contexts/AuthContext.jsx

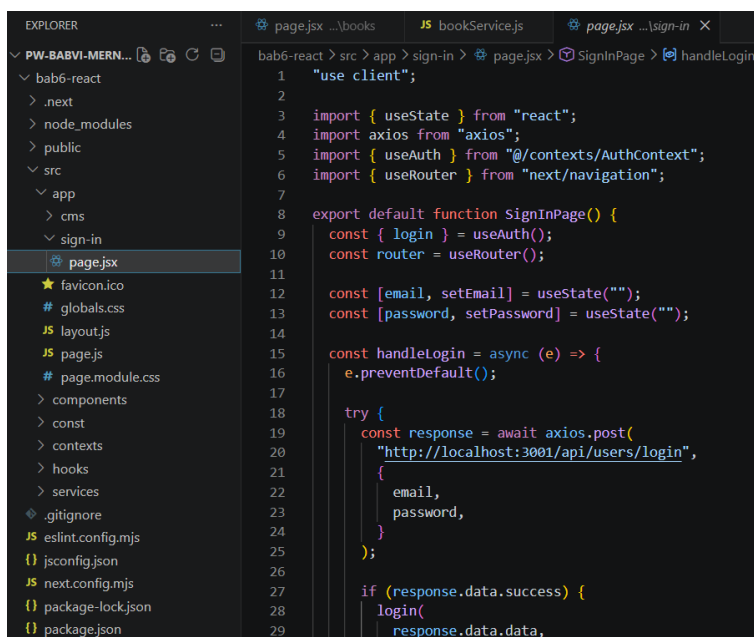


```
1 "use client";
2
3 import { createContext, useContext, useEffect, useState } from "react";
4
5 const AuthContext = createContext();
6
7 export const AuthProvider = ({ children }) => {
8   const [user, setUser] = useState(null);
9   const [loading, setLoading] = useState(true);
10
11   useEffect(() => {
12     checkAuth();
13   }, []);
14
15   const checkAuth = () => {
16     try {
17       const token = localStorage.getItem("accessToken");
18       const userData = localStorage.getItem("user");
19       const expiresIn = localStorage.getItem("expiresIn");
20
21       if (token && userData && expiresIn) {
22         const currentTime = Math.floor(Date.now() / 1000);
23
24         if (currentTime >= parseInt(expiresIn)) {
25           logout();
26           return;
27         }
28
29         setUser(JSON.parse(userData));
30       }
31     } catch (error) {
32       console.log(error);
33     }
34   };
35
36   return (
37     <AuthContext.Provider value={{ user, setUser, loading, setLoading, checkAuth }}>
38       {children}
39     </AuthContext.Provider>
40   );
41 }
```

Penjelasan:

AuthContext digunakan untuk mengelola status autentikasi pengguna, menyimpan data login, token, serta mengatur proses login dan logout.

SS 11 src/app/sign-in/page.jsx

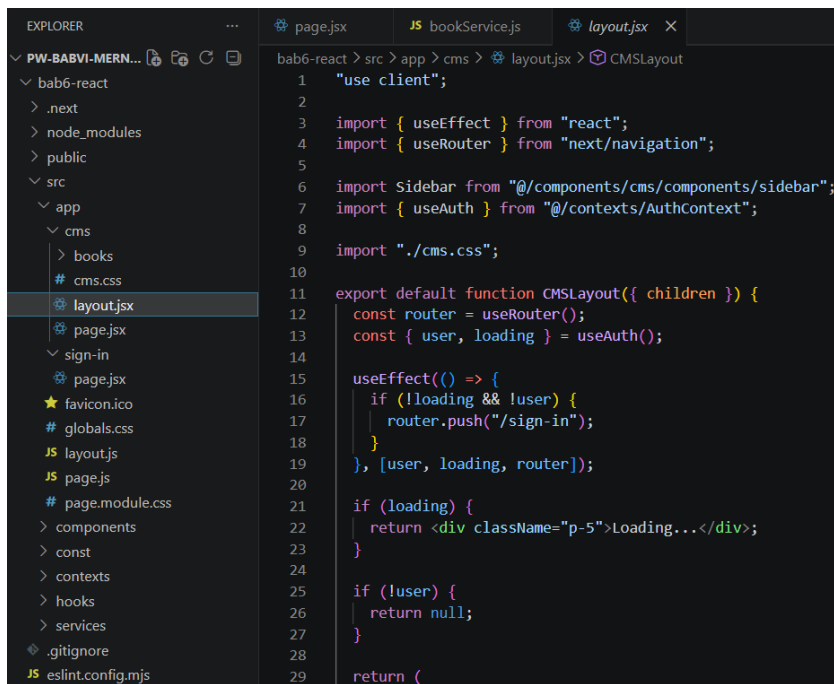


```
1 "use client";
2
3 import { useState } from "react";
4 import axios from "axios";
5 import { useAuth } from "@contexts/AuthContext";
6 import { useRouter } from "next/navigation";
7
8 export default function SignInPage() {
9   const { login } = useAuth();
10   const router = useRouter();
11
12   const [email, setEmail] = useState("");
13   const [password, setPassword] = useState("");
14
15   const handleLogin = async (e) => {
16     e.preventDefault();
17
18     try {
19       const response = await axios.post(
20         "http://localhost:3001/api/users/login",
21         {
22           email,
23           password,
24         }
25       );
26
27       if (response.data.success) {
28         login(
29           response.data.data,
30         );
31       }
32     } catch (error) {
33       console.log(error);
34     }
35   };
36
37   return (
38     <div>
39       <input type="text" value={email} onChange={setEmail} />
40       <input type="password" value={password} onChange={setPassword} />
41       <button onClick={handleLogin}>Login</button>
42     </div>
43   );
44 }
```

Penjelasan:

Halaman Sign In melakukan autentikasi ke backend menggunakan endpoint /api/users/login.

SS 12 src/app/cms/layout.jsx

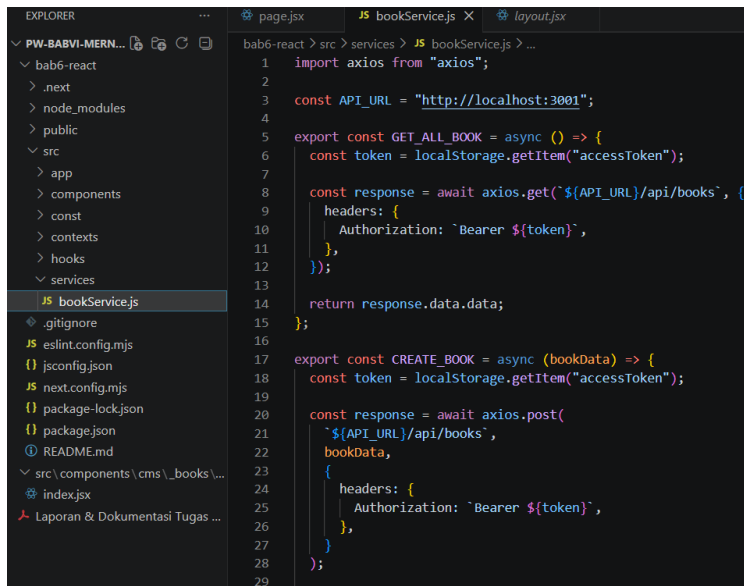


```
1  "use client";
2
3  import { useEffect } from "react";
4  import { useRouter } from "next/navigation";
5
6  import Sidebar from "@components/cms/components/sidebar";
7  import { useAuth } from "@contexts/AuthContext";
8
9  import "./cms.css";
10
11 export default function CMSLayout({ children }) {
12   const router = useRouter();
13   const { user, loading } = useAuth();
14
15   useEffect(() => {
16     if (!loading && !user) {
17       router.push("/sign-in");
18     }
19   }, [user, loading, router]);
20
21   if (loading) {
22     return <div className="p-5">Loading...</div>;
23   }
24
25   if (!user) {
26     return null;
27   }
28
29   return (
```

Penjelasan:

Layout CMS menerapkan protected route sehingga halaman CMS hanya dapat diakses oleh pengguna yang telah login.

SS 13 src/services/bookService.js

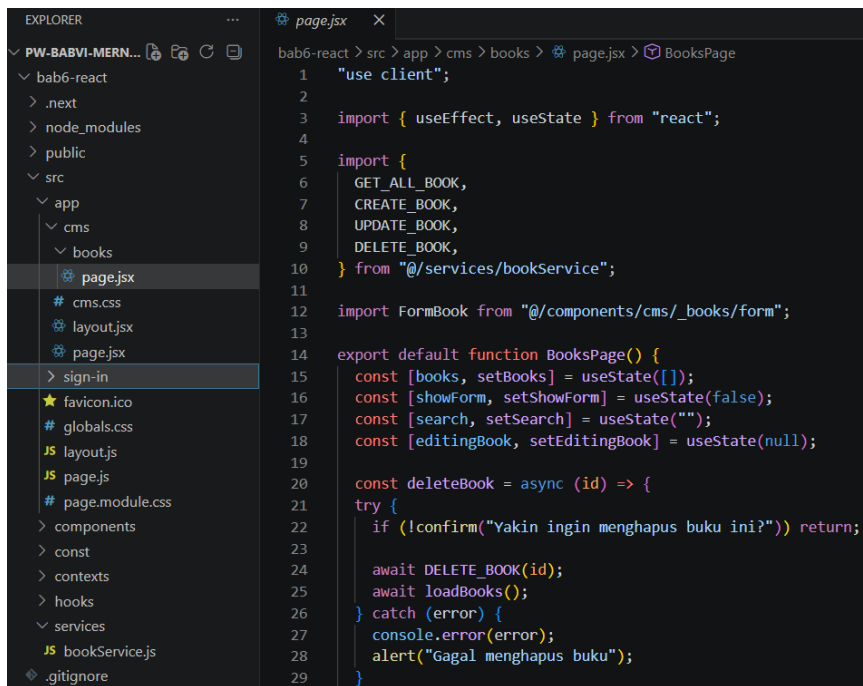


```
1  import axios from "axios";
2
3  const API_URL = "http://localhost:3001";
4
5  export const GET_ALL_BOOK = async () => {
6    const token = localStorage.getItem("accessToken");
7
8    const response = await axios.get(`${API_URL}/api/books`, {
9      headers: {
10        Authorization: `Bearer ${token}`,
11      },
12    });
13
14    return response.data.data;
15  };
16
17  export const CREATE_BOOK = async (bookData) => {
18    const token = localStorage.getItem("accessToken");
19
20    const response = await axios.post(
21      `${API_URL}/api/books`,
22      bookData,
23      {
24        headers: {
25          Authorization: `Bearer ${token}`,
26        },
27      },
28    );
29  };
```

Penjelasan:

File ini berisi fungsi untuk berkomunikasi dengan REST API Books menggunakan Axios.

SS 14 src/app/cms/books/page.jsx



```
1  "use client";
2
3  import { useEffect, useState } from "react";
4
5  import {
6    GET_ALL_BOOK,
7    CREATE_BOOK,
8    UPDATE_BOOK,
9    DELETE_BOOK,
10 } from "@services/bookService";
11
12 import FormBook from "@components/cms/_books/form";
13
14 export default function BooksPage() {
15   const [books, setBooks] = useState([]);
16   const [showForm, setShowForm] = useState(false);
17   const [search, setSearch] = useState("");
18   const [editingBook, setEditingBook] = useState(null);
19
20   const deleteBook = async (id) => {
21     try {
22       if (!confirm("Yakin ingin menghapus buku ini?")) return;
23
24       await DELETE_BOOK(id);
25       await loadBooks();
26     } catch (error) {
27       console.error(error);
28       alert("Gagal menghapus buku");
29     }
29   }
```

Penjelasan:

Halaman Books telah diintegrasikan dengan backend sehingga proses menampilkan, menambah, mengubah, dan menghapus data dilakukan melalui REST API.

4. Tampilan Aplikasi

SS 15 Halaman Login

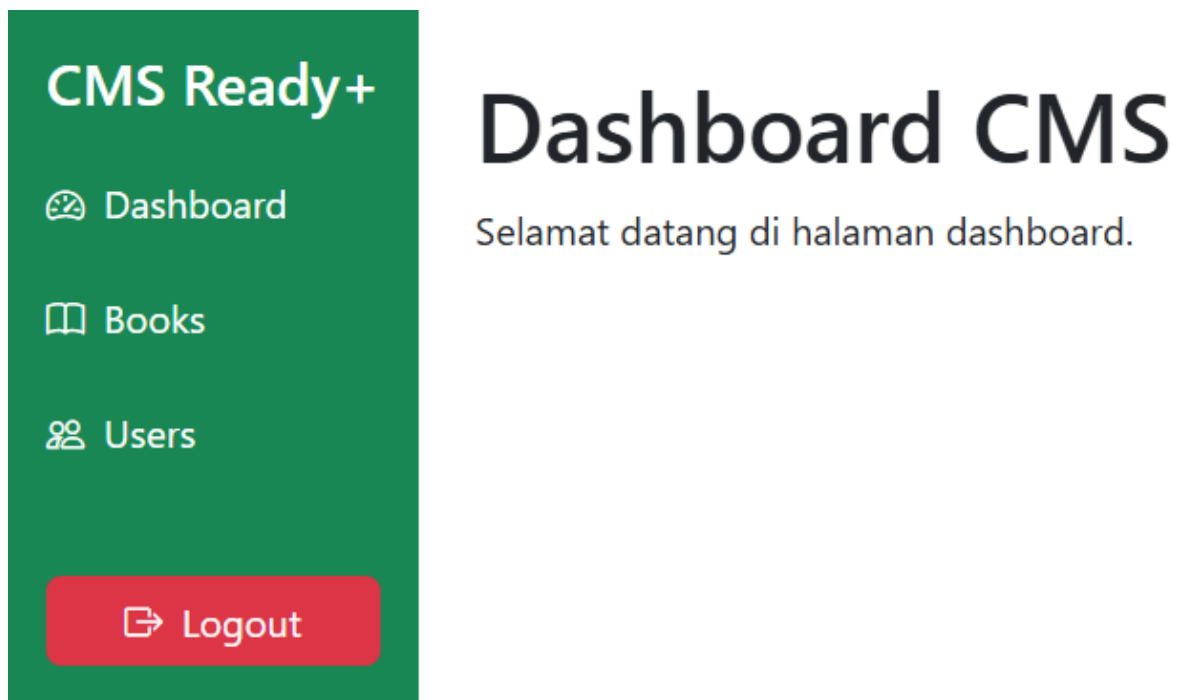
Sign In

Email

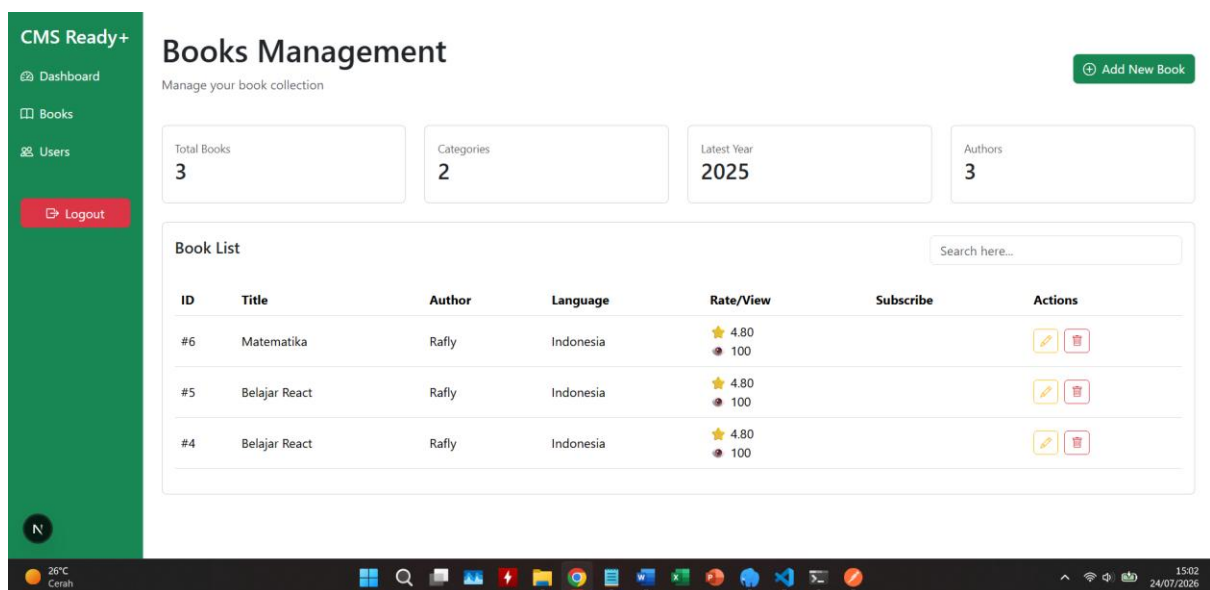
Password

Login

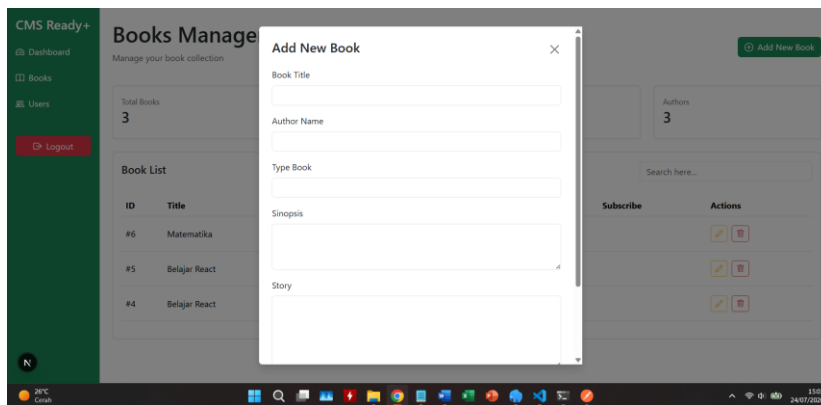
SS 16 Dashboard CMS



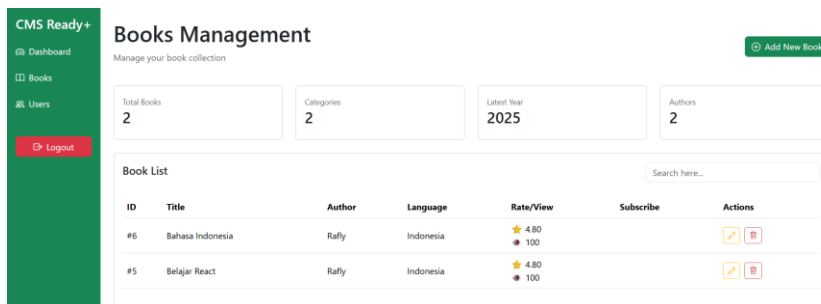
SS 17 Books sebelum Add



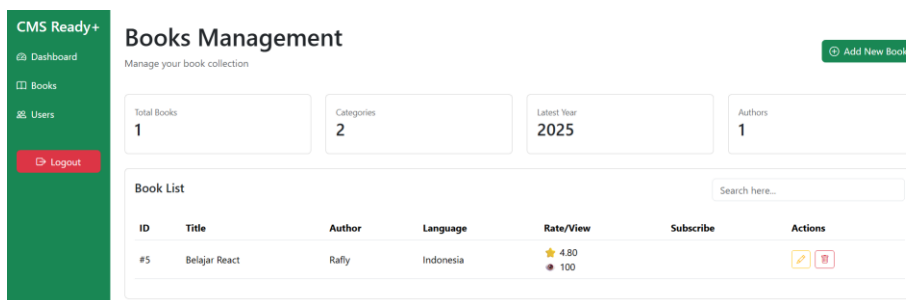
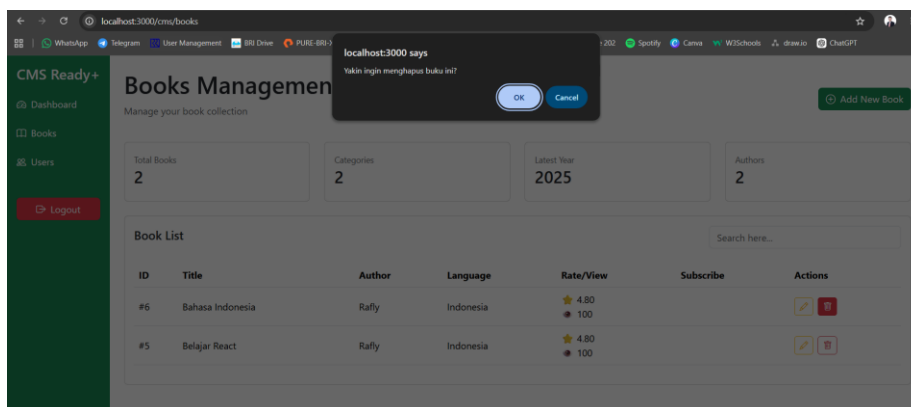
SS 18 Modal Add Book



SS 19 Data berhasil diedit



SS 20 Data berhasil dihapus



6. Kesimpulan

Berdasarkan hasil implementasi, proses autentikasi menggunakan JWT berhasil diterapkan pada aplikasi. Frontend Next.js berhasil terintegrasi dengan backend Express.js melalui REST API sehingga proses CRUD Books dapat dilakukan secara dinamis menggunakan database MySQL. Seluruh endpoint protected hanya dapat diakses oleh pengguna yang telah login menggunakan Bearer Token. Proses login, logout, penambahan, perubahan, dan penghapusan data berhasil diuji dan berjalan sesuai dengan yang diharapkan.